Neighbourhood-Aware V2X Misbehaviour Detection with Plausibility Checks and a Collusion Stress Test

Shlok (UAI05BTA2317), Jiya (UAI05BTA2313), Caleb (UAI05BTA2319) – BTech AI ML, Universal AI University
Guides: Rajesh Bhise and Laxman Buddhewad

Abstract

Connected vehicles broadcast Basic Safety Messages (BSMs) with their position, speed and heading; a vehicle that broadcasts false content can make its neighbours brake, change lanes or ignore a real hazard, and signatures cannot stop it, since they prove who sent a message, not that its content is true. Current learned detectors judge each sender alone, are evaluated on few attack types, often by accuracy alone, label every message of an attacker as malicious even while it behaves, and are rarely tested against consistent or colluding liars. For every receiving vehicle and second, a small transformer (144k parameters) classifies the 16 nearest senders over the last 10 seconds, attending over each sender's history and across the neighbouring senders. Its inputs add plausibility checks to the raw claims (kinematic consistency, a road map learned from honest traffic, exact repeats and copied trails); training adds careful, mutually consistent lies, and a receiver-side memory keeps suspicion of flagged senders. On the simulated VeReMi Extension benchmark (14 of its 19 attack types, 68,971 test windows, two seeds) it reaches precision 98.16%, recall 97.78%, F1 97.97% and accuracy 98.74% with receiver memory when an attacker is scored only in windows where it sends falsified messages, and F1 97.01% without memory when every window of an attacker vehicle counts, against 94.9% without the checks and 78.9% for the dataset's speed-consistency rule. Neighbour attention adds a small but significant 0.3 F1 points and lowers the false-positive rate from 0.90% to 0.60%, but its benefit disappears when most neighbours are attackers. Coordinated colluders with consistent lies of the default sizes were caught in at most 4% of cases before the checks and in 68–90% with them at a 1% false-positive rate, including an unseen spoofed convoy; lies that stay on the road and obey the physics remain hard (23–48%). Extended to 18 attack types including four Sybil attacks, a three-model ensemble reaches message-label F1 95.9% and accuracy 97.0%; GridSybil ghosts that look honest remain the main miss.

Index Terms: V2X, VANET, misbehaviour detection, transformer, plausibility checks, collusion, VeReMi Extension

I. The Problem

Setting. In V2X networks every vehicle periodically broadcasts a BSM with its claimed position, speed, heading and acceleration. Message signatures prove *who* sent a message but not that its *content* is true: an authenticated insider, or a vehicle with a faulty or manipulated sensor, can send false content that passes all cryptographic checks [11, 18]. Misbehaviour detection must therefore judge the content itself, on the receiving vehicle, within a fraction of a second, without ground truth about where the other vehicles really are.

Task. Let receiver r at second t hear a set of senders. We take the $K = 16$ senders closest to r by claimed distance and their claims over the last $T = 10$ seconds (messages binned to one-second steps) and output, for each sender, the probability that it misbehaves. A window is formed every 3 seconds; each sender slot is one decision. The receiver knows its own position but never the true position of a sender.

Threat model. The attacker is an insider: a vehicle with valid credentials that controls the content and the timing of its own messages. It may lie about its position, speed, heading and acceleration, replay or delay messages, copy other vehicles' content, or flood the channel. Several attackers may

coordinate (collusion). The attacker cannot change what the receiver measures about itself, and it does not control the honest vehicles. The detector runs on the receiver and sees only received messages and the receiver's own state; it has no access to roadside units, other receivers' decisions or the physical layer.

Why it is hard. Honest claims are noisy (GPS error of a few metres), vehicles stop and accelerate legitimately, and senders come and go. False alarms must stay below about 1%, since each discards an honest vehicle's data. A small, smooth lie on the road looks like an honest vehicle, and when attackers agree, the neighbours a receiver compares against are partly malicious.

Attacks. We use VeReMi Extension [2], the successor of VeReMi [1]: simulated traffic of the Luxembourg scenario with 30% misbehaving vehicles and 19 attack types. The main results use 14 of them (Table I): false positions, false speeds, a fake stop, content taken from other vehicles, delayed messages and three denial-of-service variants. An extension (Section IV-A) adds the four Sybil attacks (GridSybil, DataReplaySybil, DoSRandomSybil, DoSDisruptiveSybil), for 18 of the 19. The RandomSpeedOffset archive is excluded because it is mislabeled: both of its time slices contain the same data. The data are simulated; no claim is made about real road traffic.

TABLE I. The 14 attack types used, what the attacker changes, and the number of attacker slots in the test set (message labels).

Attack	What the attacker sends	Test slots
ConstPos	a fixed position	6,179
ConstPosOffset	true position plus a fixed offset	6,188
RandomPos	a random position each message	6,179
RandomPosOffset	true position plus a random offset	6,189
ConstSpeed	a fixed speed	6,158
ConstSpeedOffset	true speed plus a fixed offset	6,153
RandomSpeed	a random speed	6,186
EventualStop	claims to stop and stay at one position	4,556
Disruptive	content of other vehicles' messages	6,169
DataReplay	replays another vehicle's messages	6,175
DelayedMessages	its own messages, delayed	6,179
DoS	true content at a much higher rate	6,401
DoSRandom	random content at a high rate	6,402
DoSDisruptive	others' content at a high rate	6,407

What counts as misbehaviour. The benchmark labels vehicles: every message of an attacker vehicle is "malicious". Some attackers, however, send true messages for part of the time; EventualStop, for instance, drives normally until it fakes its stop. Using the per-message ground truth shipped with the dataset, we also score at the message level (Section III-F) and report both.

II. Gaps in Current Work

G1: one sender at a time. Most learned detectors classify a sender or message pair in isolation from location and movement features [5, 6, 7]. Transformer detectors have appeared [10, 9], including PAMPOS, which flags deviations from a predicted trajectory without attack labels. What a receiver gains from comparing a sender with the senders around it, and loses when those neighbours are malicious, is rarely measured.

G2: metrics. Results are often a single accuracy figure [3], which hides the false-positive rate and says nothing about detection latency.

G3: few attacks, unclear comparisons. Many studies use the original VeReMi with five position attacks [1]. The survey of Abdel Hakeem and Kim [3], our reference point, lists 97.37% for VeReMi and per-study accuracies of 95–97% [5, 6], without splits or label definitions, so such numbers are hard to compare.

G4: consistent lies and collusion. Plausibility checks [4] catch impossible claims, but attackers can fake a consistent trajectory and agree on one story. Reputation schemes assume most neighbours are honest [8]; learned detectors are rarely tested against coordinated, physically consistent lies.

G5: vehicle-level labels. Scoring every message of an attacker as malicious counts its honest periods as missed detections.

G6: where the detector runs. Recent work moves detection into federated learning to keep vehicle data private [12, 13], or into large language models at the network edge [14]; others target Sybil attacks [15, 16] or pair an LSTM detector with a reaction protocol [17]. All need a strong, cheap

local detector whose behaviour under attack is measured; that detector is our focus.

This work. One receiver-side detector and an evaluation protocol address G1–G5: a neighbour-attention ablation under growing shares of malicious neighbours (G1); full metrics, latency and significance tests (G2); 14, then 18, attack types (G3); plausibility checks, careful-lie training and a collusion stress test (G4); message- next to vehicle-level labels (G5).

III. Our Approach

Fig. 1 shows the pipeline. Raw messages heard by a receiver are grouped into windows, each claim is checked for plausibility, a transformer scores every sender, and a receiver-side memory smooths the scores over time before the alarm threshold is applied.

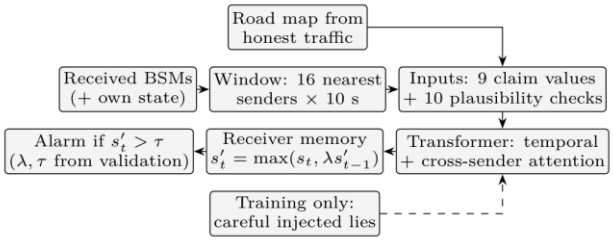


Fig. 1. Detection pipeline on one receiving vehicle.

A. Receiver-centric windows

Per sender and step we keep the raw claim: position, speed, heading, claimed acceleration along the heading, and the number of messages heard in that second. Sender identities and the dataset's simulated noise fields are never inputs. Windows are skipped when the receiver's own state is missing in any of the 10 steps.

B. Inputs: claims plus plausibility checks

For every sender and step the model receives 19 values. Nine describe the claim itself: position relative to the receiver (scaled by 300 m), speed, cosine and sine of the heading, position change since the previous step, a validity flag, and $\log(1 + \text{messages per second})$, which makes floods visible. Ten are plausibility checks computed from the claims:

- *Kinematics*: for claimed position p_t, speed v_t, heading θ_t and acceleration a_t we use a_t , $(v_t - v_{t-1}) - a_t$, $\|p_t - p_{t-1}\| - v_t$ and, when the sender moved more than 1 m, $1 - \cos(\angle(p_t - p_{t-1}) - \theta_t)$. Each is zero for a consistent honest vehicle up to sensor noise.
- *Road map*: from the receivers' own positions in the training hour we build a 5 m grid of where vehicles actually drive (675×667 cells) with the dominant driving direction per cell. Inputs are the on-road density at the claimed position and the alignment of the claimed heading with the road. The map stands in for the digital map a vehicle carries; no test data are used to build it.
- *Neighbours*: distance from the claimed position to the nearest other sender's claimed position at the same step.
- *Exact repeats*: whether the claimed position equals (to 1 cm) one of the sender's own earlier claims, and whether it equals a position that another sender claimed earlier (a copied trail); plus the logarithm of the distance moved, so centimetre-level movement is visible. In the dataset's GPS traces honest positions always jitter: none of 1.38 million honest steps had zero movement, while frozen and replayed claims repeat exactly.

Identities (18-attack extension). Senders are tracked by pseudonym, which is all a receiver sees, so the fake identities of a Sybil attacker are separate senders; labels come from the true vehicle. Three identity inputs are added: the age of the identity (following pseudonym changes), the distance at which an identity took over from one that fell silent (handover distance), and lockstep, the rigidity of a sender's offset to the other sender that moves most closely with it. The 18-attack model therefore has 22 inputs per sender and step.

C. Model

A factorised transformer encodes the window (Fig. 2): a linear embedding ($d = 64$) with a

learned time encoding, then two pre-normalised blocks of (i) temporal self-attention of a sender over its own 10 steps, (ii) spatial self-attention across the senders present at the same step (the neighbourhood) and (iii) a feed-forward layer; 4 heads, dropout 0.1. The newest step feeds a two-layer head; padding and missing steps are masked. The ablation without neighbour attention restricts the spatial attention to the sender itself, so both have 144k parameters.

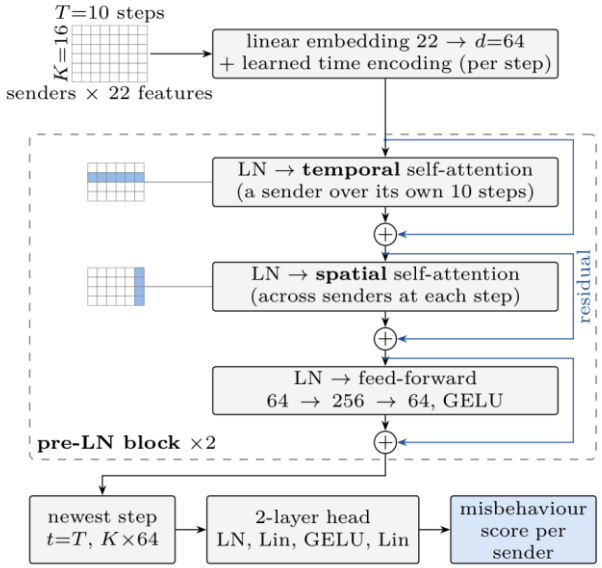


Fig. 2. Model architecture: one pre-normalised block, applied twice, mixes each sender's own history (temporal attention) and the senders heard at the same step (spatial, or neighbour, attention); the head scores every sender from its newest step. The ablation masks spatial attention to the sender itself. Inputs: 22 = 9 claim values + 10 checks + 3 identity inputs (19 in the 14-attack model).

D. Training with careful lies

The benchmark's attacks are mostly physically implausible. To teach the model consistent lies as well, we add a copy of half of the training windows (those with ≥ 2 honest senders) in which 1–4 honest senders are rewritten as liars, coordinated or independent: a position offset of 20–300 m that keeps the true motion, or a fake stop (position frozen with GPS-like jitter, speed zero, acceleration consistent with the braking). A third lie type, a convoy driving at the receiver, is *held out* of training to test generalisation. Checks are recomputed on the rewritten claims. This adds 39,691 windows with about 74,000 liar slots.

E. Receiver memory

A receiver that has flagged a sender keeps a decaying suspicion of it: $s'_t = \max(s_t, \lambda s'_{t-1})$ over that receiver's windows of the same sender, forgotten after 30 s of silence. λ and the alarm threshold are chosen on validation data.

F. Message-level labels

A received message is *falsified* if its claimed position differs from the sender's true position at reception by more than 1 m or its speed by more than 0.5 m/s, using the ground-truth trace. Honest claims match the ground truth exactly, so the tolerance only absorbs rounding. An attacker slot is positive if the sender sent at least one falsified message in the window; DoS attackers, whose misbehaviour is the flood itself, are always positive. Attacker slots without any falsified message are left out of message-level scoring. They are 2.0% of all attacker slots, almost all from EventualStop before its stop (26% of its slots).

G. Implementation

The detector is written in Python with PyTorch, NumPy and scikit-learn. A loader parses the VeReMi Extension logs (one JSON file per vehicle, Zenodo record 20090854, CC-BY 4.0), reading message type 2 (the receiver's own state) and type 3 (received BSMs) and taking labels from the file names; 250 receivers are sampled evenly per simulation slice. Features are stored as half-precision memory-mapped files (about 1 GB of GPU memory). Algorithm 1 summarises detection and training, and Table II the settings. Code, models, logs and table scripts are in the project repository (entry points `run_v2.py`, `label_active.py`, `run_analysis_v2.py`, `run_collusion_v2.py`). All random choices use fixed seeds; rebuilding our first model's test windows on a second machine reproduced its numbers exactly.

Algorithm 1 Receiver-side detection and training

Part 1: detection (each receiver r, every 3 s; K=16, T=10)
$\mathcal{B} \leftarrow$ BSMs heard in the last T seconds, grouped by pseudonym
$\mathcal{S} \leftarrow$ the K senders nearest to r by *claimed* position
$W \leftarrow K \times T$ raw claims: position, speed, heading, accel., msgs/s
$X \leftarrow W$ plus checks: kinematics; road map; nearest other claim; exact repeat; copied trail; identity age, handover, lockstep
$s \leftarrow \sigma(\text{Transformer}(X)) \in [0,1]^K$ ▷ one score per sender
for all sender $j \in \mathcal{S}$ **do**
  $s'_j \leftarrow \max(s_j, \lambda\, s'_{j,\text{prev}})$ ▷ memory; reset after 30 s silence
  flag j if $s'_j > \tau$
end for
Part 2: training (first simulated hour only)
Road map $\leftarrow$ 5 m grid of vehicles' own positions, headings
$\mathcal{D} \leftarrow$ windows (lines 1–4) + a copy of half of the windows with $\geq$ 2 honest senders, 1–4 of them made careful liars (offset 20–300 m or stop)
Train 8 epochs: BCE (pos. weight 2), AdamW, one-cycle LR
Pick τ, $\lambda \in \{.9, .95, .97, .99\}$ by max. F1 on validation

Algorithm 1: Receiver-side detection and training

TABLE II. Implementation and training settings.

Data	VeReMi Extension, 14 attack subsets, 14:00–16:00
Split	hour 1: train; hour 2: 30% val. / 70% test receivers
Windows	109,823 train (+39,691 lies), 29,547 val., 68,971 test
Window	$K = 16, T = 10$ steps of 1 s, stride 3 s, 250 receivers/slice
Inputs	19 per sender and step (9 claim values, 10 checks)
Road map	5 m grid, 675×667 cells, own positions (hour 1)
Model	transformer, $d = 64$, 4 heads, 2 blocks, 144k parameters
Loss	weighted BCE (positive weight 2), batch 128, 8 epochs
Optimiser	AdamW, lr 2×10^{-3}, weight decay 10^{-2}, one-cycle
Hardware	laptop RTX 3050 Ti (4 GB), 16 GB RAM; 10–15 min each
Software	Python 3.12, PyTorch 2.5, NumPy, scikit-learn
Seeds	2 per model; threshold and memory decay from validation
18 attacks	mean of both and a $d = 96$, 3-block model (470k, 16 ep.)

IV. Evaluation

Protocol. Models are trained on the first simulated hour of each of the 14 attack subsets and tested on the second hour, whose receivers are split into validation and test blocks (Table II). All thresholds are chosen on validation. We report precision, recall, F1, accuracy and false-positive rate (FPR), each the mean of two training seeds (the seeds differ by at most 0.5 points). Per-attack F1 scores an attack against the honest senders of its own simulation run. Differences between models are tested with a paired bootstrap over receivers (2,000 resamples) and Holm correction across attacks. Baseline rules are the range check and the speed-consistency check of VeReMi [1], applied to the same windows.

TABLE III. VeReMi Extension, 14 attack types, 68,971 test windows, mean of 2 seeds (%). Message labels count an attacker as positive only in windows where it sent a falsified message (ground truth); vehicle labels count every window of an attacker vehicle. Thresholds and memory decay are picked on validation. Reference: 97.37% reported for VeReMi (original, 5 attacks) in the survey of Abdel Hakeem and Kim (2025).

Labels	Model	Precision	Recall	F1	Accuracy	FPR
message	Ours	98.65	97.12	97.88	98.69	0.60
message	No nbr. attention	97.99	97.11	97.54	98.48	0.90
message	No lie augm.	98.75	97.39	98.07	98.80	0.56
message	Ours + memory	98.16	97.78	97.97	98.74	–
message	Speed check (rule)	96.21	68.51	80.03	89.34	1.22
vehicle	Ours	98.65	95.41	97.01	98.14	0.60
vehicle	No nbr. attention	97.87	95.48	96.66	97.91	0.96
vehicle	No lie augm.	98.75	95.68	97.19	98.25	0.56
vehicle	Ours + memory	98.16	96.09	97.12	98.20	–
vehicle	Speed check (rule)	95.64	67.21	78.94	88.67	1.41

TABLE IV. Injected consistent lies (careful: acceleration consistent, GPS-like jitter), m colluders per window, coordinated; colluder recall at a matched 1% validation FPR, mean of 2 seeds. Training saw offset and stop lies only. Before the plausibility checks both models caught at most 4% at these sizes. In the two on-road rows the last two columns are single runs (seed 0).

Scenario	Ours	No nbr. attention	No lie augm.
Offset 50 m, $m = 1$	0.85	0.83	0.81
Offset 50 m, $m = 8$	0.83	0.83	0.65
Fake stop, $m = 1$	0.69	0.69	0.01
Fake stop, $m = 8$	0.72	0.70	0.01
Convoy (unseen), $m = 1$	0.87	0.86	0.86
Convoy (unseen), $m = 8$	0.90	0.90	0.72
Offset 50 m along road, $m = 4$	0.25	0.24	0.20
Convoy on own road, $m = 4$	0.42	0.42	0.41

TABLE V. Recall on real attackers by the share of attacker vehicles among the senders in the window (message labels, mean of 2 seeds). Last row: recall gain of neighbour attention computed within each attack type (number of attack types with at least 200 attacker slots in the bin).

Attacker share (%)	0–20	20–40	40–60	60–100
Windows	22,310	22,583	15,161	8,917
No nbr. att. recall	0.978	0.974	0.970	0.963
No nbr. att. FPR (%)	0.98	0.93	0.70	1.01
Ours recall	0.979	0.975	0.971	0.959
Ours FPR (%)	0.68	0.64	0.36	0.77
Within-attack gain	+0.002 (4)	+0.001 (14)	+0.001 (14)	-0.004 (14)

Overall (Table III). With message labels the detector reaches F1 97.88% (recall 97.12%); with receiver memory F1 is 97.97%, and precision, recall, F1 and accuracy are all above 97.4% in both seeds. With vehicle labels F1 is 97.01% (97.12% with memory): recall falls because the honest-looking windows of attackers count as misses. Our first model, with the same architecture but only the nine raw inputs, reached F1 94.9% (precision 98.4%, recall 91.6%, accuracy 96.9%, vehicle labels); the plausibility checks raised ConstPosOffset recall from 0.68 to 0.96 and EventualStop recall on active windows to 0.99. The speed-consistency rule reaches F1 80.0% and fails completely on constant position offsets, EventualStop, delayed messages and DoS (F1 below 0.1, also on the 18-attack test set in Fig. 4). Mean time to the first alarm is 0.9 s (median 0, counted from the first window that contains the sender, which already holds up to 10 s of history); 96% of attacker–receiver pairs are eventually flagged.

Neighbour attention. Attending across senders adds 0.3 F1 points (95% bootstrap interval about [0.2, 0.4] in both seeds and both label schemes), is significantly better on 8–10 of the 14 attacks and significantly worse on none, and lowers the FPR from 0.90% to 0.60%. Its benefit depends on the neighbourhood (Table V): within each attack type, the gain shrinks as the share of attacker vehicles in the window grows and turns negative (-0.004) once attackers are the majority. In the earlier model without checks the same pattern was stronger (from $+0.071$ to -0.023). Neighbour context helps while most neighbours are honest; it is not a defence against a malicious majority.

Consistent and colluding lies (Table IV). We inject lies into the test windows, recompute all inputs, and compare models at the same 1% validation FPR. Before the checks, both our models caught at most 4% of such colluders at these sizes, coordinated or not (larger offsets were caught more often, Fig. 3); neighbour attention even let coordinated convoys raise false alarms on honest neighbours from 0.9% to 3.0%. With the checks, a 50 m offset is caught in 83–86% of cases (63% at 25 m, 98% at 400 m), a fake stop in about 70%, and the held-out convoy in 87–90%. Careful-lie training makes fake stops detectable (1% without it) and costs 0.2 F1 points on benchmark attacks.

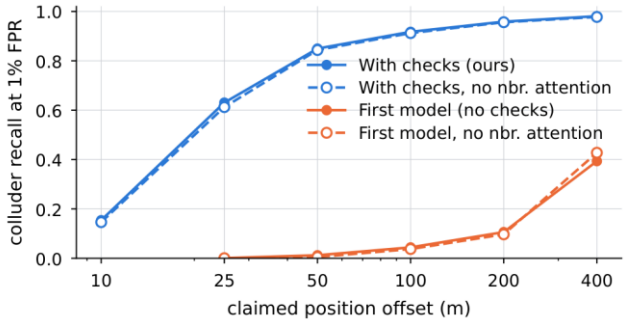


Fig. 3. Recall on four coordinated colluders that shift their claimed position, as a function of the shift, at a matched 1% validation false-positive rate (mean of 2 seeds). Solid: with neighbour attention; dashed: without.

Two map-aware attacks remain hard: shifting the claimed position 50 m *along* the road (23–25%) and a convoy that follows the receiver's own road (36–48%).

Reference numbers. The survey [3] lists 97.37% for VeReMi (accuracy, precision, recall and F1 reported together) and F1 98% for positional attacks with a reputation-based detector [8]; So *et al.* report precision 95.91% and detection rate 83.73% with physical-layer checks [4]. With receiver memory our message-level results exceed 97.37% on all four measures (without it, recall is 97.12%; position attacks alone: F1 97.5–99.4%). These figures come from a different dataset (original VeReMi, five attacks) and partly undocumented protocols; they place our results but are not a like-for-like comparison.

A. Extension to Sybil attacks

We add the four Sybil attacks (18 of 19 attack types; 141,260 training windows plus 51,033 with injected lies, 88,695 test windows) and the three identity inputs. Outside the Sybil runs every vehicle uses one pseudonym, so the 14-attack windows are unchanged. Ghost messages differ from the true position in 74–100% of the inspected cases (honest messages never), so message labels mark all DoS Sybil and 96% of GridSybil attacker slots active. The final system averages three models (the two 144k-parameter models and one wider, deeper model with $d = 96$, 3 blocks and 16 epochs), threshold picked on validation.

TABLE VI. Extension to 18 of the 19 attack types (the 14 above plus four Sybil attacks), 88,695 test windows. Ours: mean score of three models; no-neighbour rows: mean of two models (seeds 0, 1); thresholds picked on validation; per-attack F1 in Fig. 4.

Overall	Precision	Recall	F1	Accuracy
Ours, message labels	96.30	95.46	95.88	97.04
No nbr. attention, message	96.28	92.69	94.45	96.07
Ours, vehicle labels	96.33	94.44	95.38	96.66
No nbr. attention, vehicle	96.31	91.69	93.94	95.68

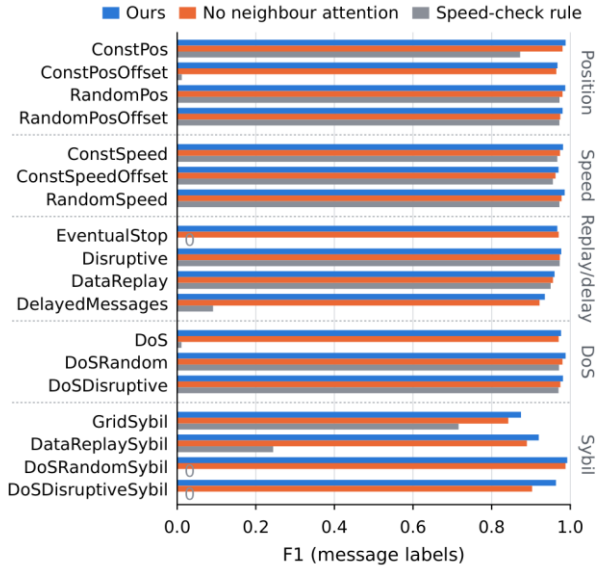


Fig. 4. Per-attack F1 on the 18-attack test set (message labels): ours (three-model ensemble), the two-model ensemble without neighbour attention, and the speed-consistency rule; a grey 0 marks a rule F1 of exactly zero.

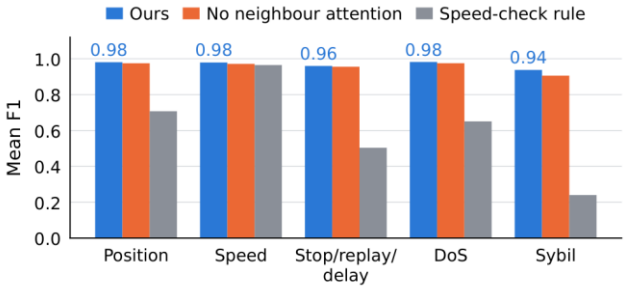


Fig. 5. Mean F1 per attack family (18-attack test set, message labels, attacks weighted equally); values shown for ours.

On message labels (Table VI) the FPR is 2.07%, and all four measures are *below* the 97.37% reference, unlike the 14-attack results. Neighbour attention matters more here: without it F1 is 94.45% (+1.3 points comparing two-model ensembles, +1.2 to +1.4 per seed, against +0.3 on 14 attacks). Per attack (Fig. 4), the three-model ensemble scores above the two-model ablation everywhere except EventualStop (F1 0.967 against 0.971), and the gap is widest on the Sybil family (mean F1 0.938 against 0.906, Fig. 5), where the speed rule reaches 0.24. GridSybil, the largest attack (20,598 test slots), is the weak spot with recall 0.82 (F1 0.87); DataReplaySybil reaches recall 0.91, DoSDisruptiveSybil 0.97 and DoSRandomSybil 0.997. On the 14 original runs it scores F1 about 97.5% at FPR 1.4% (0.6% for the 14-attack model): it lowers its threshold to catch ghosts. A bigger model alone did not help (F1 95.19% against 95.76% for the two-seed ensemble of small models), nor did receiver memory, likely because ghosts change pseudonyms. Missed GridSybil ghosts look honest on every content check (on the road, consistent motion), and neither arrival-time coincidence nor formation rigidity separated them: the content limit again.

V. Discussion

What the detector relies on. Our first model saw the same senders but only the raw claims and caught only physically impossible ones (positions beyond radio range, speed that does not match movement). The checks turn knowledge a vehicle already has (a map, the laws of motion, the jitter of measured positions) into explicit inputs; the largest gains come from attacks that are consistent in

themselves. Neighbour attention is a second-order effect that lowers false alarms, probably because an unusual honest vehicle (one stopping hard) is easier to accept when its neighbours behave the same way; it helps when neighbours are mostly honest and becomes a liability when they are not, so a deployed system should not treat agreement among neighbours as proof.

Deployment. The 14-attack model (144k parameters) needs well under a millisecond per window, so a vehicle could score every incoming window; the 18-attack ensemble (about 760k parameters) needs roughly four times the compute. In practice the road map must come from map data, the alarm threshold should be set per deployment, and the receiver memory delays the release of a falsely flagged sender by a few seconds, fine for misbehaviour reports, not for discarding safety-critical messages.

An adaptive attacker who knows the checks will keep its lies on the road and consistent with its claimed motion, and jitter frozen positions. Our map-aware stress test models this attacker (23–48% caught), and missed GridSybil ghosts behave the same way. Content checks did not close this gap; closing it needs independent evidence of where the sender is (signal strength or angle of arrival, its own camera and radar, roadside units).

VI. Limitations and Conclusion

The benchmark is simulated; the main results cover 14 of 19 attack types and the Sybil extension 18 (below the 97.37% reference); the 14-attack results average two seeds and the 18-attack system is a three-model ensemble; dense rush-hour traffic was evaluated only for the earlier model; the road map assumes the receiver has a map of the area; and the collusion test rewrites honest vehicles of the trace without re-selecting the 16 senders after the lie. Message-level labels are our explicit choice; vehicle-level results are reported alongside.

Within these limits, plausibility checks computed from the claims did most of the work: they moved the detector from F1 94.9% to 97.0% under vehicle labels (97.9% with message labels) and made consistent lies mostly detectable. Neighbour attention gives a small, significant gain that vanishes once most neighbours are malicious. On-road, physically consistent lies (23–48% caught) and GridSybil ghosts (18-attack F1 95.9%) need independent evidence of the sender's position (signal strength, on-board sensors, roadside units); it and the newer VeReMi NextGen data are our next steps.

References

[1] R. W. van der Heijden, T. Lukaseder, and F. Kargl, "VeReMi: A dataset for comparable evaluation of misbehavior detection in VANETs," in *Security and Privacy in Communication Networks (SecureComm)*. Cham, Switzerland: Springer, 2018, pp. 318–337.

[2] J. Kamel, M. Wolf, R. W. van der Heijden, A. Kaiser, P. Urien, and F. Kargl, "VeReMi Extension: A dataset for comparable evaluation of misbehavior detection in VANETs," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, doi: 10.1109/ICC40277.2020.9149132.

[3] S. A. Abdel Hakeem and H. Kim, "Advancing intrusion detection in V2X networks: A comprehensive survey on machine learning, federated learning, and edge AI for V2X security," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 8, pp. 11137–11205, Aug. 2025.

[4] S. So, J. Petit, and D. Starobinski, "Physical layer plausibility checks for misbehavior detection in V2X networks," in *Proc. 12th Conf. Security and Privacy in Wireless and Mobile Networks (WiSec)*, 2019, pp. 84–93.

[5] S. So, P. Sharma, and J. Petit, "Integrating plausibility checks and machine learning for misbehavior detection in VANET," in *Proc. 17th IEEE Int. Conf. Machine Learning and Applications (ICMLA)*, 2018, pp. 564–571.

[6] P. K. Singh, S. Gupta, R. Vashistha, S. K. Nandi, and S. Nandi, "Machine learning-based approach to detect position falsification attack in VANETs," in *Security and Privacy*. Singapore: Springer, 2019, pp. 166–178.

[7] A. Sharma and A. Jaekel, “Machine learning based misbehaviour detection in VANET using consecutive BSM approach,” *IEEE Open J. Veh. Technol.*, vol. 3, pp. 1–14, 2022.

[8] S. Gyawali, Y. Qian, and R. Q. Hu, “Machine learning and reputation based misbehavior detection in vehicular communication networks,” *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8871–8885, Aug. 2020.

[9] K. Kalogiannis, A. M. Hussain, and P. Papadimitratos, “PAMPOS: Causal transformer-based trajectory prediction for attack-agnostic misbehavior detection in V2X networks,” arXiv:2605.06833, 2026.

[10] H. Li, K. Kalogiannis, A. M. Hussain, and P. Papadimitratos, “AttentionGuard: Transformer-based misbehavior detection for secure vehicular platoons,” arXiv:2505.10273, 2025.

[11] S. Ruj, M. A. Cavenaghi, Z. Huang, A. Nayak, and I. Stojmenovic, “Data-centric misbehavior detection in VANETs,” arXiv:1103.2404, 2011.

[12] E. Mármol Campos, A. González Vidal, J. L. Hernández Ramos, and A. Skarmeta, “Federated learning for misbehaviour detection with variational autoencoders and Gaussian mixture models,” arXiv:2405.09903, 2024.

[13] U. Demir, Y. E. Sagduyu, T. Erpek, H. Jafari, S. Kompella, and M. Xue, “Distributed federated learning for vehicular network security: Anomaly detection benefits and multi-domain attack threats,” arXiv:2505.23706, 2025.

[14] W. Hamhoum and S. Cherkaoui, “MistralBSM: Leveraging Mistral-7B for vehicular networks misbehavior detection,” arXiv:2407.18462, 2024.

[15] M. Thomas, A. Borah, and A. Paranjothi, “Trust-aware Sybil attack detection for resilient vehicular communication,” *Internet Technology Letters*, 2024, arXiv:2411.07520.

[16] M. S. T. Khan, M. S. A. Utsho, and M. Jahan, “A robust framework for Sybil attack detection in vehicular ad hoc networks,” arXiv:2606.11667, 2026.

[17] M. Franceschini, “A neural-network based anomaly detection system and a safety protocol to protect vehicular network,” Master’s thesis, arXiv:2411.07013, 2024.

[18] A. Boualouache and T. Engel, “A survey on machine learning-based misbehavior detection systems for 5G and beyond vehicular networks,” arXiv:2201.10500, 2022.